

RefNest 2.0

底层架构重构方案

Architecture Redesign Proposal

事件驱动 • 仓储模式 • 契约化 IPC • 同步就绪 • 可测试

本方案针对《RefNest vs Zotero 框架对比分析》中识别的七项架构短板，
提出一套完整的分层重构设计。方案兼容全部现有功能，
为所有待扩展功能（Future Work）预留清晰的落位点，
并给出分阶段迁移路线图，避免一次性重写风险。

Contents

1	重构目标与设计原则	3
1.1	要解决的七个问题	3
1.2	五条设计原则	3
2	分层架构总览	3
2.1	五层结构	4
2.2	重构后目录结构	4
3	数据模型重构：仓储 + 事件总线	5
3.1	三级职责分离	5
3.2	Service 层写入范式	5
3.3	Notifier 事件总线	6
4	契约化 IPC 与安全加固	6
4.1	单一契约源	6
4.2	文件访问白名单（pathGuard）	7
4.3	二进制传输改造	7
5	数据库层：Worker 线程 + 事务 + 迁移框架	8
5.1	DB Worker 线程	8
5.2	UnitOfWork 事务封装	8
5.3	版本化迁移框架	8
6	渲染进程：查询缓存与失效键	8
6.1	queryCache 机制	8
6.2	乐观更新	9
6.3	Zustand 职责收缩	9
7	同步就绪设计（Sync-ready）	9
7.1	三项预埋	10
7.2	未来同步引擎的接入点	10
8	托管存储布局（StorageManager）	10
8.1	现状问题	10
8.2	目标布局（对齐 Zotero storage key 模式）	10
9	通用任务队列（JobQueue）与 AI 管线	11
9.1	从 pdf2md 专用队列泛化	11
10	测试策略	11
11	现有功能与 Future Work 落位总表	12
11.1	现有功能映射	12
11.2	Future Work 落位	13
12	分阶段迁移路线图	13

13 重构前后对比总结

14

1 重构目标与设计原则

1.1 要解决的七个问题

对照《框架对比分析》的评分结论，本方案逐项给出对策：

#	现有短板（评分）	对策（章节）
1	数据模型无变更通知（4/10）	Notifier 事件总线 + 仓储层（§3）
2	状态管理各自 reload（3/10）	查询缓存 + 失效键机制（§6）
3	IPC 无校验、路径无限权（6/10）	契约化 IPC + zod 校验 + 路径白名单（§4）
4	同步 SQL 阻塞主进程（5/10）	DB Worker 线程 + 事务封装（§5）
5	测试覆盖近零（1/10）	分层可测设计 + 测试金字塔（§9）
6	同步系统缺失（2/10）	版本号 + 墓碑 + 操作日志预埋（§7）
7	附件路径为绝对路径	托管存储布局 + 相对路径（§8）

1.2 五条设计原则

设计原则

1. 本地优先（**Local-first**）：所有操作先落本地 SQLite，同步是叠加层而非前提。

2. 单一写入口（**Single write path**）：所有数据写操作必须经过 Service 层，禁止 UI 或 IPC 处理器直接拼 SQL。写入自动触发事件广播。

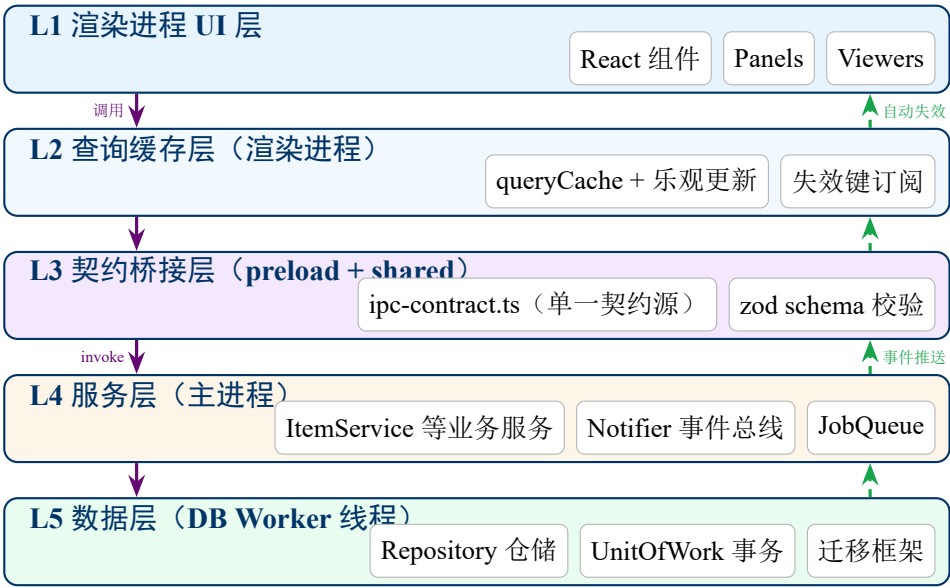
3. 契约先行（**Contract-first IPC**）：主/渲染进程共享一份类型化 IPC 契约文件，参数与返回值均有 schema，两端编译期对齐。

4. 同步就绪（**Sync-ready**）：每次写入递增版本号、软删除留墓碑、写操作追加到操作日志——即使同步功能暂不实现，数据结构先行。

5. 渐进迁移（**Strangler pattern**）：新旧代码并存，按模块逐步替换，每个阶段都保持应用可运行、可发布。

2 分层架构总览

2.1 五层结构



左侧紫色箭头是请求流（UI 发起 → 数据层执行），右侧绿色虚线是事件流（数据变更 → 自动广播 → UI 自动刷新）。现有架构只有请求流没有事件流，这是本次重构的核心补足。

2.2 重构后目录结构

```
src/
├── shared/                                # 两端共享（编译进 main 和 renderer）
│   ├── ipc-contract.ts                  # ❶ IPC 契约：通道名 + zod schema + 类型
│   ├── events.ts                        # ❷ Notifier 事件类型定义
│   └── types.ts                          # 领域模型类型（现有，保留）
├── main/
│   ├── index.ts                         # 入口（瘦身：只做装配）
│   ├── ipc/
│   │   ├── gateway.ts                   # ❸ IPC 网关：统一校验 + 错误包装 + 日志
│   │   └── handlers/                   # 按域拆分的薄处理器（只调 Service）
│   ├── services/                        # ❹ 业务服务层（单一写入口）
│   │   ├── ItemService.ts
│   │   ├── AttachmentService.ts
│   │   ├── CollectionService.ts
│   │   ├── TagService.ts
│   │   ├── ImportService.ts            # PDF/BibTeX/CSL-JSON 导入编排
│   │   ├── KeywordService.ts          # 关键词提取（MD→PDF→CrossRef 三级）
│   │   └── SettingsService.ts
│   ├── core/
│   │   ├── Notifier.ts                  # ❺ 事件总线（进程内 + IPC 推送）
│   │   ├── JobQueue.ts                  # ❻ 通用任务队列（取代 pdf2md 专用队列）
│   │   └── StorageManager.ts            # ❼ 托管存储布局（附件目录管理）
│   └── db/
│       ├── worker.ts                   # ❶ DB Worker 线程入口
│       ├── DbClient.ts                  # ❷ 主进程侧异步代理（postMessage RPC）
│       ├── migrations/                  # ❸ 版本化迁移目录（001_xxx.ts ...）
│       ├── UnitOfWork.ts                # ❹ 事务封装
│       └── repositories/                # ❺ 仓储（纯 SQL，无业务逻辑）
```

	ItemRepo.ts	
	AttachmentRepo.ts	
	...	
	OplogRepo.ts	# 回 操作日志（同步预埋）
integrations/		# 外部 API 客户端（无状态、可 mock）
mineru.ts		# 现 mineruApi.ts
crossref.ts		# 现 crossref.ts
localServer.ts		# 现 server.ts（浏览器扩展）
security/		
pathGuard.ts		# 回 文件访问白名单
preload/		
index.ts		# 由 ipc-contract 自动生成桥接
renderer/src/		
data/		# 回 查询缓存层
queryCache.ts		# 缓存 + 失效键 + 乐观更新
hooks.ts		# useItems / useAttachments / ...
stores/		# 仅存纯 UI 状态（选中、视图类型）
components/		# 现有组件（数据获取改走 hooks）

标 回 的是新增模块，其余为现有代码的迁移落位。

3 数据模型重构：仓储 + 事件总线

3.1 三级职责分离

现有代码中 db/items.ts 混合了 SQL、业务规则和 IPC 返回格式。重构为：

层	职责	禁止事项
Repository	纯 SQL 读写，行 ↔ 对象映射	不含业务规则，不发事件
Service	业务规则、事务编排、发事件	不写 SQL，不碰 IPC
IPC Handler	参数校验、调 Service、包装错误	不含任何逻辑（≤5 行）

3.2 Service 层写入范式

以「更新文献标题」为例，所有写操作遵循同一模板：

```
1 // services/ItemService.ts
2 class ItemService {
3   async updateItem(id: number, patch: Partial<ItemData>): Promise<Item> {
4     return this.uow.transaction(async (tx) => {
5       const before = await tx.items.getById(id)
6       if (!before) throw new NotFoundError('item', id)
7
8       // 1. 写主表：版本号自动 +1, updated_at 自动刷新
9       const after = await tx.items.update(id, patch)
10
11       // 2. 追加操作日志（同步预埋，见第7节）
12       await tx.oplog.append('item', id, 'modify', patch)
13
14       // 3. 事务提交后广播事件（提交前不发，防止脏通知）
15       tx.onCommit(() =>
```

```

16     this.notifier.emit({ type: 'item.modified', ids: [id] }))
17
18     return after
19   })
20 }
21 }

```

对策 #1: 解决「无变更通知」

任何写操作必然经过 Service → 必然发事件 → UI 必然收到刷新信号。「忘记 reload」这类 bug 在结构上不可能发生。

3.3 Notifier 事件总线

借鉴 Zotero.Notifier, 但用 TypeScript 类型化事件:

```

1 // shared/events.ts -- 事件即契约, 两端共享
2 export type DomainEvent =
3   | { type: 'item.created';      ids: number[] }
4   | { type: 'item.modified';    ids: number[] }
5   | { type: 'item.trashed';     ids: number[] }
6   | { type: 'item.deleted';     ids: number[] }
7   | { type: 'attachment.changed'; itemIds: number[] }
8   | { type: 'tag.changed';      itemIds: number[] }
9   | { type: 'collection.changed'; ids: number[] }
10  | { type: 'job.progress';      job: JobStatus } // pdf2md 等
11
12 // main/core/Notifier.ts
13 class Notifier {
14   emit(e: DomainEvent): void {
15     for (const fn of this.subscribers) fn(e) // 进程内订阅者
16     this.mainWindow?.webContents.send('domain-event', e) // 推给渲染进程
17   }
18   subscribe(fn: (e: DomainEvent) => void): () => void { ... }
19 }

```

进程内订阅者包括: 未来的同步引擎、全文索引器、插件系统——它们只需订阅事件, 无需侵入业务代码。

4 契约化 IPC 与安全加固

4.1 单一契约源

现状: 通道名散落在 ipc.ts、preload/index.ts、env.d.ts 三处, 靠人工保持一致。重构为单文件契约:

```

1 // shared/ipc-contract.ts
2 import { z } from 'zod'
3
4 export const contract = {
5   'items.update': {
6     args: z.tuple([z.number().int().positive(), ItemPatchSchema]),
7     returns: ItemSchema,
8   },
9   'fs.readFile': {
10    args: z.tuple([z.string().max(1024)]),

```

```

11     returns: z.instanceof(Uint8Array),    // 见 4.3
12   },
13   // ... 全部 40+ 通道
14 } as const
15
16 export type Contract = typeof contract

```

主进程网关和 preload 桥接均由此文件驱动：

```

1 // main/ipc/gateway.ts -- 统一注册，自动校验
2 for (const [channel, spec] of Object.entries(contract)) {
3   ipcMain.handle(channel, async (_e, ...raw) => {
4     const args = spec.args.parse(raw)    // zod 校验，失败即拒绝
5     try {
6       return { ok: true, data: await handlers[channel](...args) }
7     } catch (err) {
8       log.error(channel, err)
9       return { ok: false, error: toPublicError(err) } // 不泄露堆栈
10    }
11  })
12 }

```

对策 #3a：解决「IPC 无校验」

新增通道必须先契约文件声明 schema，否则两端编译报错。校验、错误包装、日志在网关统一处理，处理器本体归零样板代码。

4.2 文件访问白名单 (pathGuard)

```

1 // main/security/pathGuard.ts
2 export function assertReadable(p: string): string {
3   const real = realpathSync(resolve(p))    // 消解 .. 和符号链接
4   const allowed = [
5     app.getPath('userData'),
6     settings.getStoragePath(),
7   ].filter(Boolean).map(d => realpathSync(d))
8   if (!allowed.some(dir => real.startsWith(dir + sep)))
9     throw new AccessDeniedError(p)
10  return real
11 }
12 // 所有 fs.* IPC 处理器入口第一行调用 assertReadable / assertWritable

```

对策 #3b：解决「任意文件读取」

realpathSync 先行，杜绝 ..\..\ 和符号链接逃逸；白名单仅含 userData 与用户指定存储根目录。

4.3 二进制传输改造

现有 `fs.readFile` 返回 `number[]`（每字节一个 JS Number，10 MB 图片膨胀为约 80 MB 堆对象）。Electron 的结构化克隆原生支持 `Uint8Array` 零拷贝传输，直接替换即可；大文件（PDF 本体）则不走 IPC，改由 `refnest-file://` 协议流式响应，并在协议 handler 中同样接入 `pathGuard`。

5 数据库层：Worker 线程 + 事务 + 迁移框架

5.1 DB Worker 线程

保留 better-sqlite3（其同步 API 在专用线程内反而是优点：无回调开销、天然串行），但整体移入 worker_threads：



```

1 // 主进程侧 API 形态不变，只是变为真异步：
2 const items = await db.items.getAll() // 不再阻塞 GUI 事件循环
  
```

对策 #4：解决「同步 SQL 阻塞主进程」

万条级全量查询、FTS 重建、批量导入均在 Worker 内执行，主进程事件循环零阻塞。列表 UI 配合分页 + 虚拟滚动（见 §6.3）。

5.2 UnitOfWork 事务封装

```

1 // 批量导入 50 篇 PDF：要么全部成功，要么整体回滚
2 await uow.transaction(async (tx) => {
3   for (const meta of parsedEntries) {
4     const item = await tx.items.create(meta)
5     await tx.creators.setForItem(item.id, meta.creators)
6     await tx.collectionItems.add(collectionId, item.id)
7   }
8 }) // 内部映射为 BEGIN IMMEDIATE ... COMMIT / ROLLBACK
  
```

5.3 版本化迁移框架

```

1 // db/migrations/003_add_oplog.ts
2 export default defineMigration({
3   version: 3,
4   up(db) {
5     db.exec(`CREATE TABLE oplog (
6       seq INTEGER PRIMARY KEY AUTOINCREMENT,
7       object_type TEXT NOT NULL, object_id INTEGER NOT NULL,
8       op TEXT NOT NULL, payload TEXT,
9       ts INTEGER NOT NULL, synced INTEGER DEFAULT 0)`);
10   },
11 })
12 // 运行器：读 user_version → 顺序执行缺失迁移 → 每步独立事务
13 // 幂等保障：版本号判断取代 IF NOT EXISTS 猜测
  
```

6 渲染进程：查询缓存与失效键

6.1 queryCache 机制

渲染进程新增轻量查询缓存（约 150 行，无需引入 React Query 依赖）：

```

1 // renderer/src/data/hooks.ts
2 export function useAttachments(itemId: number) {
3   return useQuery(
4     ['attachments', itemId], // 缓存键
5     () => window.refnest.attachments.getByItem(itemId))
6 }
7
8 // renderer/src/data/queryCache.ts -- 订阅主进程事件流
9 window.refnest.onDomainEvent((e) => {
10   switch (e.type) {
11     case 'attachment.changed':
12       invalidate(['attachments', ...e.itemIds]); break
13     case 'tag.changed':
14       invalidate(['tags', ...e.itemIds])
15       invalidate(['items']) // 列表内联标签也要刷新
16       break
17     case 'item.modified':
18       invalidate(['items']); invalidate(['item', ...e.ids]); break
19   }
20 })

```

对策 #2: 解决「各组件手动 reload」

组件只声明「我要什么数据」, 不再负责「何时刷新」。AttachmentsTab / TagsTab / MetadataTab 中所有手写 `reload()` 与 `useEffect` 依赖追踪全部删除。MinerU 转换完成后注册附件 → `attachment.changed` 事件 → 打开中的附件面板自动出现新的.md 与图片文件夹, 无需用户切换刷新。

6.2 乐观更新

高频小操作（加标签、改标题）先改缓存后发请求, 失败回滚:

```

1 mutate(['tags', itemId],
2   (old) => [...old, newTag], // 立即反映到 UI
3   () => window.refnest.tags.setForItem(itemId, names)) // 失败自动还原

```

6.3 Zustand 职责收缩

`itemStore` 不再持有 `items` 数据 (移交 `queryCache`), 只保留纯 UI 状态: `selectedId`、`activeCollection`、`viewerPath/viewerType`、`searchQuery`、排序方式。数据与视图状态彻底分离。列表组件配合虚拟滚动 (渲染可视区约 30 行), 万条文献流畅滚动。

7 同步就绪设计 (Sync-ready)

同步引擎本身属 Future Work, 但数据结构必须现在就位, 否则届时需要全量数据迁移。

7.1 三项预埋

机制	设计
版本号	<code>items.version</code> 已存在但从未递增 → Service 层每次写入自动 +1, 作为未来冲突检测依据 (对齐 Zotero 协议语义)
墓碑	永久删除不再 DELETE 行, 改写 <code>tombstones</code> 表 (<code>object_type, key, ts</code>), 使删除操作可同步到其他设备
操作日志	<code>oplog</code> 表记录每次写操作 (见 §5.3), 同步引擎按 <code>synced=0</code> 增量上传; 本地回放也可用于撤销功能

7.2 未来同步引擎的接入点

```
1 // 同步引擎作为 Notifier 订阅者接入, 零侵入业务代码:
2 class SyncEngine {
3   constructor(notifier: Notifier) {
4     notifier.subscribe((e) => this.scheduleUpload()) // 防抖批量上传
5   }
6   async pull() {
7     // 远端变更 → Service 层写入 (复用同一写入口) → 自动广播 → UI 自动刷新
8   }
9 }
```

同步目标可插拔: Zotero WebDAV、自建服务器、S3 兼容存储, 均实现同一 SyncBackend 接口。

8 托管存储布局 (StorageManager)

8.1 现状问题

附件 `path` 存绝对路径, 且 MinerU 输出直接写在源 PDF 旁边: 用户移动 PDF、换电脑、改盘符, 所有附件全部失联; 也无法做文件同步。

8.2 目标布局 (对齐 Zotero storage key 模式)

<storage.path>/	
└─ storage/	
└─ <item-key>/	# 每篇文献一个 8 位 key 目录
└─┬─ paper.pdf	# 导入时复制 (或移动) 进来
└─┬─ paper.md	# MinerU 输出
└─┬─ images/	# 精准解析图片目录
└─┬─ annotations.json	# 未来: 注释层持久化

数据库中 `attachments.path` 改存相对路径 `storage/<key>/paper.pdf`, 运行时由 StorageManager 拼接存储根目录。好处: 换机器只需拷贝 storage 目录 + 数据库文件; 文件同步按 item-key 目录为单位; `imagedir`、多版本.md 天然归位。

迁移兼容

提供一次性「整理文件到托管存储」向导: 复制现有附件进 storage 布局并改写路径; 同时保留「链接模式」(仅记路径不复制) 供拒绝托管的用户使用, 与 Zotero

的 linked file 模式对齐。

9 通用任务队列（JobQueue）与 AI 管线

9.1 从 pdf2md 专用队列泛化

现有串行队列是 pdf2mdService 内的专用实现（设计良好但不可复用）。泛化为通用 JobQueue，现有与未来的长任务统一接入：

任务类型	状态	说明
pdf2md.agent	现有，迁入	MinerU 免费模式
pdf2md.precision	现有，迁入	MinerU 精准模式（ZIP+ 图片）
keywords.extract	现有，迁入	MD→PDF→CrossRef 三级提取
fulltext.index	新增	将.md 全文纳入 FTS（当前仅索引标题摘要）
embedding.build	Future	文献向量化，支撑语义检索 / RAG 问答
webarchive.save	Future	浏览器扩展网页存档
sync.batch	Future	同步引擎批量上传/下载

```
1 interface Job<T = unknown> {
2   id: string; type: string; payload: T
3   priority: number           // 用户手动触发 > 导入自动触发
4   attempts: number          // 失败重试（指数退避，上限 3 次）
5 }
6 class JobQueue {
7   // 按 type 配置并发度：pdf2md 串行(1)，keywords 可并行(4)
8   // 进度统一走 Notifier 的 job.progress 事件（取代专用 pdf2md:status 通道）
9   // 队列持久化到 jobs 表：应用崩溃/重启后未完成任务自动恢复
10 }
```

队列持久化解决现有痛点：转换进行中关闭应用，任务丢失且无提示。

10 测试策略

分层架构使每层可独立测试，按测试金字塔投入：

层级	对象与工具	要点
单元测试（多）	纯函数：关键词提取、splitKeywords、路径工具、pathGuard、zod schema、vitest	无 mock 即可测，先行覆盖
仓储测试（中）	Repository + 迁移框架。vitest + 内存 SQLite (:memory:)	真 SQL 不 mock；迁移幂等性（空库/旧库各跑一次）
服务测试（中）	Service 层。vitest + 内存库 + 假 Notifier	断言「写入后必发正确事件」
集成测试（少）	IPC 网关 + 外部 API 客户端。mock MinerU/Cross-Ref HTTP	契约 schema 拒绝非法参数
E2E（极少）	Playwright for Electron	冒烟：导入 PDF → 列表出现 → 打开查看器

对策 #5：解决「零测试覆盖」

现有代码难测的根因是层混杂（SQL + 业务 + Electron API 纠缠）。分层后 Service 与 Repository 均不依赖 Electron，普通 Node 环境即可测试，CI（GitHub Actions）跑全部非 E2E 用例，PR 必须绿灯。

11 现有功能与 Future Work 落位总表

11.1 现有功能映射

现有功能	重构后落位
PDF 导入 + CrossRef 元数据	ImportService 编排；CrossRef 客户端移入 integrations/（可 mock）
MinerU Agent / Precision 转换	JobQueue 任务类型；MinerU 客户端移入 integrations/
多模态 Markdown 查看器	组件不变；图片改走 refnest-file:// 流式协议 + pathGuard
图片画廊（imagedir）	组件不变；目录纳入托管存储 storage/<key>/images/
关键词提取 + 标签	KeywordService（JobQueue 接入）+ TagService；提取器纯函数化，单测覆盖
集合树 / 回收站	CollectionService + 墓碑化删除
FTS 搜索	保留 FTS5；新增 fulltext.index 任务把.md 全文纳入索引
本地服务器 + 浏览器扩展	integrations/localServer.ts，写操作改调 Service（获得事件广播）
设置系统 / i18n / 原生菜单	SettingsService 统一（含 API Token 改用 safeStorage 加密存储）

11.2 Future Work 落位

待扩展功能	架构支撑点
多设备同步	§7 三项预埋 + SyncEngine 订阅 Notifier + SyncBackend 可插拔接口
PDF 注释持久化	storage/<key>/annotations.json + AnnotationService + annotation.changed 事件
引文插入（citeproc）	CitationService 读 ItemRepo；纯计算，无需新基建
语义检索 / RAG 问答	embedding.build 任务 + 向量存 SQLite（sqlite-vec 扩展）；问答服务订阅 item.created 自动增量索引
网页存档	webarchive.save 任务；产物存 storage/<key>/snapshot/
Agent 模式图片支持	JobQueue 内组合任务：Agent 转文本 + 本地 pdf.js 抽图，产物布局与 Precision 对齐
多窗口 / 浮动阅读器	Notifier 已按 window 广播设计，第二窗口天然收到全部事件；仅需窗口管理器
插件系统	插件 = Notifier 订阅者 + 受限 Service API 白名单；契约化 IPC 天然提供权限边界
撤销 / 重做	oplog 逆操作回放（§7 副产品）

12 分阶段迁移路线图

遵循 Strangler 模式，每阶段结束应用均可正常发布：

阶段	内容	工作量	风险
P0	pathGuard + Uint8Array 传输 + Token 加密（纯加固，不动结构）	1–2 天	低
P1	契约化 IPC：ipc-contract.ts + 网关 + preload 自动生成	3–4 天	低（形态不变）
P2	Notifier + Service 层（先 Item/Attachment/Tag 三域）+ queryCache，删除全部手动 reload	5–7 天	中
P3	DB Worker 线程 + UnitOfWork + 迁移框架	4–5 天	中
P4	JobQueue 泛化 + 队列持久化；同步预埋（oplog/墓碑/版本号）	3–4 天	低
P5	托管存储布局 + 迁移向导	4–5 天	中（涉及用户文件）
测试	随各阶段同步编写（P1 起 CI 强制）	贯穿	–

总计约 4–5 周。P0–P2 完成后即解决对比分析中危害最大的三项（安全漏洞、状态混乱、无事件通知）；P3–P5 面向规模与未来功能。

迁移纪律

- 每阶段独立 PR 到 `feat/pdf-annotation`，禁止跨阶段大杂烩提交
- 旧模块删除前，新旧路径并行跑通一个发布周期
- 数据库迁移（P3/P5）前自动备份 `userData` 下的 `.sqlite` 文件

13 重构前后对比总结

维度	现状	重构后
写数据路径	IPC 处理器直连 SQL	唯一入口：Service + 事务 + 事件
UI 刷新	各组件手动 reload	事件驱动自动失效，含乐观更新
IPC 安全	无校验、任意路径读取	zod 契约 + pathGuard 白名单
DB 执行	主进程同步阻塞	Worker 线程，主进程零阻塞
长任务	pdf2md 专用队列，不持久	通用 JobQueue，崩溃可恢复
附件路径	绝对路径，易失联	托管存储 + 相对路径，可迁移可同步
同步能力	仅空表预留	版本号 + 墓碑 + oplog 就位，引擎即插即用
可测试性	层混杂，难以测试	每层独立可测，CI 强制
预期评分	4.3 / 10	约 7.5 / 10（同步引擎落地后 8+）

一句话总结

本方案不更换任何技术栈（Electron / React / better-sqlite3 全部保留），只补齐 Zotero 用 20 年验证过的三件核心基建——单一写入口、事件总线、同步就绪的数据结构——并以渐进迁移保证每一步都可发布、可回退。